

데이터베이스

Database

목차

- ▶ SQL이란?
- ▶ SQL 문법

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The shapes are concentrated on the right side of the image, with some extending towards the left.

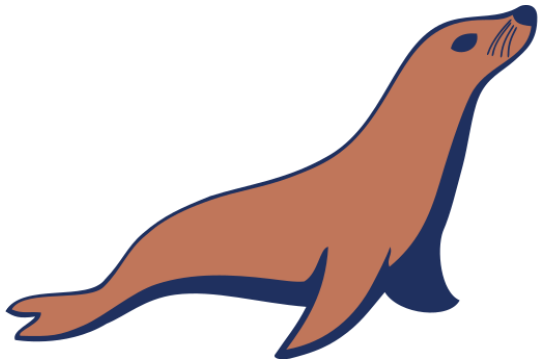
SQL

SQL이란?

- ◆ Structured Query Language의 줄임말로 DB를 다루기 위한 언어
- ◆ DB와 SQL은 땔래야 땔 수 없는 존재이지만 DB는 SQL인 것은 아니다.
(DB != SQL)
- ◆ 때문에 DB를 다룰 줄 아는 것과 SQL을 다룰 줄 아는 것은 깊이에서 차이가 존재한다.
- ◆ SQL은 생각보다 직관적인 표현으로 난이도가 쉬운 편이나 깊이는 더 깊은 편.
- ◆ 앞선 DBMS들 모두가 공통적으로 SQL 문법이 같으나 작은 부분에서 다르기 때문에 주의해야 한다.

SQL이란?

◆ MariaDB



MariaDB

- ◆ 오픈소스 관계형 DBMS.
- ◆ MySQL과 동일한 소스 코드를 기반으로 하지만, MySQL이 오라클 소유가 되고 오라클이 라이선스 상태를 변경하면서 이에 반발하여 MySQL개발자가 개발한 DBMS이다.
- ◆ 때문에 MySQL을 사용하는 방식을 다소 공유할 수 있다.

MariaDB

◆ 설치 순서

- `sudo apt update`
- `sudo apt install mariadb-server`

◆ 실행 순서

- 백그라운드에 MariaDB Server 실행(실제 동작 및 관리)
`sudo systemctl start mariadb` /or/ ... `mysql`(하위 호환성)
재시작시 자동 시작 등록
`sudo systemctl enable mariadb` /or/ ... `mysql`(하위 호환성)
옵션을 수정할 경우 `service`를 재가동해야함.
- MariaDB Client 실행(명령어 수행 및 UI)
`sudo mariadb`
- 4. DB 생성 `CREATE DATABASE {DB이름};`
- 5. DB 접속 `use {DB이름};`

MariaDB

- ◆ `show databases;`
: 현재 서버에 있는 DB를 확인하는 DBMS 명령어
- ◆ `use {DB이름};`
: 해당 DB를 활성화 시킴.
- ◆ 활성화된 DB 확인 방법

```
MariaDB [(none)]> _
```

최초 상태

```
Database changed  
MariaDB [test]> _
```

use 명령어 입력 옆에 DB이름이 떠있음)

- ◆ 그 외 명령어는 'help;'를 통해 확인 가능

명령어의 차이

- ◆ 터미널을 이용해서 **DB**를 사용하게 되면 총 3가지 명령어의 차이를 이해해야 한다.
- ◆ 'Linux 커널용 명령어'[ex) apt, dpkg, systemctl]
: **DB**에 접속하지 않은 상태에서 사용하는 명령어, **Linux** 명령어이기 때문에 **DB**에 들어가 있을 때는 작동하지 않는다.
- ◆ **MariaDB** 명령어[ex) use, show 등]
: **MariaDB** 프로그램만의 자체 명령어, **DB**에 접속했을 때만 가능한 명령어로, 모든 **DB**가 공용하지 못하고, **MariaDB**(하위 호환이라면 **Mysql**까지)에서만 사용 가능. ';'이 필수는 아니다.
- ◆ **SQL**[ex) CREATE, SELECT 등]
: **DB**들이 공통적으로 사용하는 데이터용 명령어, **DB**구분 없이 사용하지만 **DB**에 접속한 상태에서만 사용가능하며, **DB**마다 조금씩 다른 부분이 존재할 순 있다.(대동소이) 항상 ';'으로 라인을 종료한다.

SQL문법

SQL을 만든 이유

- ◆ 데이터 저장방식은 테이블을 이용하는 방식으로 성장하는 것이 가장 효율적이라 발전했지만 이를 다루는 용어는 **DB**마다 달랐습니다.
- ◆ 이를 '불편한 일'이라 규명한 과거 개발자들은 '이런 동일한 형태를 다루는 명령어를 통일하자!'라는 의도로 만든 것이 **SQL**입니다.
- ◆ **SQL**은 데이터를 조작/제어하기 위해 만들어졌으며, 덕분에 우리는 **DB**마다 다른 명령어를 쓰지 않고도 일관적으로 데이터를 다룰 수 있게 되었습니다.
- ◆ 하지만 **DB**의 발전과 함께 **DB**의 자체 기능을 위해 **SQL**을 다르게 사용하거나 정의를 빼거나 넣음으로서 큰 흐름에서는 같지만 세부 흐름에서는 조금 다른 방식이 생겨났습니다.
- ◆ 결국, 개발자의 편의를 위해 **SQL**이 생겨난 것이니, **SQL**을 배우는 것을 스트레스를 받아선 안됩니다.

SQL 기본 문법의 가닥

◆ DML(Data Manipulation Language, CRUD)

- CREATE(생성)
: DB 혹은 TABLE, TUPLE을 생성하는 행동
앞선, CREATE 명령어에 해당
- READ(조회)
: 원하는 데이터를 요구에 맞게 검색 및 조회하는 행동
SELECT문에 해당
- UPDATE(수정)
: 주어진 DATA를 수정하는 행동
INSERT 및 UPDATE문에 해당
- DELETE(삭제)
: 주어진 DATA를 삭제하는 행동
DROP 및 DELETE문에 해당

SQL 코드 이해하기(별첨)

- ◆ 기본적으로 기본형은 훨씬 복잡함.
- ◆ 자주 만날 수 있는 것을 기준으로 기본형을 구축하여 보여주는 것이므로 지금 것이 '정답'이 아님을 꼭 인지 할 것
- ◆ {}나 []가 없는 것은 의미적으로 '그대로 써야하는 문법'을 의미
- ◆ {}나 [] 내부의 값은 '그런 의미의 값을 채우라'라는 뜻이므로 그대로 적으면 안된다.
- ◆ {}는 '꼭 필요한 내용'을 의미, 작성시에는 {} 포함하지 않음
- ◆ []는 '조건부에 따라 사용'을 의미, 작성시에는 [] 포함하지 않음

SQL 기본 문법 예시 입력

- ▶ `CREATE TABLE DEPT (DEPTNO DECIMAL(2), DNAME VARCHAR(14), LOC VARCHAR(13),
CONSTRAINT PK_DEPT PRIMARY KEY (DEPTNO));`
- ▶ `INSERT INTO DEPT VALUES (10,'ACCOUNTING','NEW YORK'), (20,'RESEARCH','DALLAS'),
(30,'SALES','CHICAGO'), (40,'OPERATIONS','BOSTON');`
- ▶ `CREATE TABLE EMP (EMPNO DECIMAL(4), ENAME VARCHAR(10), JOB VARCHAR(9), MGR
DECIMAL(4), HIREDATE DATE, SAL DECIMAL(7,2), COMM DECIMAL(7,2), DEPTNO DECIMAL(2),
CONSTRAINT PK_EMP PRIMARY KEY (EMPNO), CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO)
REFERENCES DEPT(DEPTNO));`
- ▶ `INSERT INTO EMP VALUES (7369,'SMITH','CLERK',7902,'1980-12-17',800,NULL,20),
(7499,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30),
(7521,'WARD','SALESMAN',7698,'1981-02-21',1250,500,30),
(7566,'JONES','MANAGER',7839,'1981-04-02',2975,NULL,20),
(7654,'MARTIN','SALESMAN',7698,'1981-09-28',1250,1400,30),
(7698,'BLAKE','MANAGER',7839,'1981-05-01',2850,NULL,30),
(7782,'CLARK','MANAGER',7839,'1981-06-09',2450,NULL,10),
(7788,'SCOTT','ANALYST',7566,'1987-07-13',3000,NULL,20),
(7839,'KING','PRESIDENT',NULL,'1981-11-17',5000,NULL,10),
(7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30),
(7876,'ADAMS','CLERK',7788,'1987-07-13',1100,NULL,20),
(7900,'JAMES','CLERK',7698,'1981-12-03',950,NULL,30),
(7902,'FORD','ANALYST',7566,'1981-12-03',3000,NULL,20),
(7934,'MILLER','CLERK',7782,'1982-01-23',1300,NULL,10);`

데이터를 조작하는 SQL : SELECT

- ▶ `SHOW COLUMNS FROM {Table명};`
: 입력된 Table의 속성값을 확인하는 명령어. 줄여서 'DESC {Table명}'
- ▶ `SELECT {Column명} FROM {Table명} [추가조건];`
: `SELECT * FROM DEPT; // SELECT * FROM EMP;`
: `SELECT DEPTNO FROM DEPT; // SELECT ENAME FROM EMP;`
: `SELECT EMPNO, ENAME, JOB FROM EMP;`
: `SELECT * FROM DEPT WHERE DNAME = 'RESEARCH';`
: `SELECT * FROM EMP WHERE EMPNO = 5;`
: `SELECT * FROM EMP WHERE SAL > 1000; /OR/ SAL != 1100;`
: `SELECT * FROM EMP WHERE ENAME > 'B'; /OR/ ENAME > 'ALLEN';`
: `SELECT * FROM EMP WHERE HIREDATE > '1981-01-01'; /OR/`
: `'1981.01.01';`
: `SELECT * FROM EMP WHERE ENAME LIKE '%NE%';`

데이터를 조작하는 SQL : SELECT

- ▶ LIKE 문법

- ▶ ‘%’

: 문자의 형태와 길이를 제한하지 않는 와일드카드. 특정 문자 앞에 있을 경우 그 문자 앞으로만 적용되며 반대로도 가능

EX) `SELECT * FROM EMP WHERE ENAME LIKE '%AR%';`

- ▶ ‘_’

: 문자의 형태는 상관없지만 길이는 1개로 제한하는 와일드카드. %와 똑같이 사용되나 _하나당 한문자로 매칭시킬 수 있다.

EX) `SELECT * FROM EMP WHERE NAME LIKE '%AR_';`

- ▶ `SELECT * FROM EMP WHERE ENAME LIKE '__NE%';`

데이터를 조작하는 SQL : SELECT

- ▶ AND, OR, NOT, XOR
- ▶ `SELECT * FROM EMP WHERE ENAME = 'ALLEN' AND SAL = 1100;`
: 매칭 결과가 없으니까, Empty.
- ▶ `SELECT * FROM EMP WHERE ENAME = 'ALLEN' OR SAL = 1100;`
- ▶ `SELECT * FROM EMP WHERE ENAME NOT LIKE '%AR%';`
- ▶ `SELECT * FROM EMP WHERE ENAME = 'ALLEN' XOR SAL = 1100;`
`SELECT * FROM EMP WHERE ENAME = 'ALLEN' XOR SAL = 1600;`

데이터를 조작하는 SQL : SELECT

- ▶ ORDER BY 문법

: 생각과 결과와는 다르게 데이터는 순서대로 저장되지 않는다.
기본형 >>

```
SELECT {Column명} FROM {Table명} [ORDER BY {Column명}  
[추가 조건]];
```

- ▶ SELECT * FROM EMP ORDER BY ENAME; /OR/ ORDER BY JOB, SAL;

- ▶ SELECT * FROM EMP ORDER BY HIRDEATE;
: 시간값 또한 비교가 가능하다.

- ▶ SELECT * FROM EMP ORDER BY ENAME DESC;

- ▶ SELECT * FROM EMP LIMIT 5;

- ▶ SELECT * FROM ORDER BY ENAME LIMIT 5;
: 문법상 ORDER BY 보다 LIMIT이 더 뒤로 와야한다.

데이터를 조작하는 SQL : SELECT

▶ 검색 범위 지정 OFFSET, LIMIT

- LIMIT : 최대 출력을 조정
- OFFSET : 시작지점을 조정, LIMIT과 다르게 단독 사용 불가
- 특정 범위의 데이터를 불러올 때 사용됨.
- 데이터의 형태 확인을 위해서 부분만 불러올 때 사용됨.
- `SELECT * FROM EMP ORDER BY ENAME LIMIT 3 OFFSET 3;`
: 또한 순서에 주의
- `SELECT * FROM EMP WHERE SAL > 1100 ORDER BY ENAME LIMIT 3 OFFSET 3;`

데이터를 조작하는 SQL : SELECT

▶ SELECT문 실행 순서

순서	문법	작동
1	FROM	테이블 지정
2	ON	조인 조건 확인
3	JOIN	테이블 조인(병합)
4	WHERE	조건식
5	GROUP BY	데이터 그룹화
6	HAVING	그룹화 조건식
7	SELECT	속성 선택
8	DISTINCT	중복 제거
9	ORDER BY	데이터 정렬
10	LIMIT	데이터 출력 제한
11	OFFSET	데이터 시작 지점 선정

데이터를 조작하는 SQL : INSERT

- ▶ INSERT INTO {Table명} [Column명] VALUES (속성값1, 속성값2, ...),
(...), ...;
: INSERT INTO EMP VALUES
(7369, 'SMITH', 'CLERK', 7902, '1980-12-17', 800, NULL, 20),
(7499, 'ALLEN', 'SALESMAN', 7698, '1981-02-20', 1600, 300, 30);
: 앞선 입력 값으로 확인, 순서를 지킨다면 Column명은 적지 않아도 됨
- ▶ 새로운 값을 추가
: INSERT INTO EMP (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM,
DEPTNO) VALUES (7793, 'SAM', 'ANALYST', 7782, '1983-03-03', 2800,
NULL, 20);
: SELECT * FROM EMP;
- ▶ INSERT INTO EMP (EMPNO) VALUES (7795);
: 기본값이 존재할 경우 값이 없어도 넣을 수 있다.
- ▶ INSERT를 제대로 쓰기 위해서 '자료형(DataType)'에 대한 이해가 동반되어야 합니다.

데이터를 조작하는 SQL : 자료형

- ▶ 프로그래밍 언어 이상으로 자료형이 중요함.
- ▶ 가능하면 최소한의 크기의 자료형을 사용하는 것이 좋음
Ex) 예/아니오를 선택한 항목을 저장하는데 **INT**보다는 **Bool**값이 좋음.
- ▶ 이에 따라, 사전에 어떠한 정보를 어떻게 저장할 지에 대한 고민을 해야함.
- ▶ 물론, 경우에 따라서 이 자료형을 수정하는 것이 가능하지만 최소한으로 진행해야한다. 만약 **DB**에서 바뀌게되면 이와 연동된 프로그램들에서도 수정을 해야하는 소요가 생기기 때문이다.

데이터를 조작하는 SQL : 자료형

▶ 정수형

- **INT** : -2147483648~2147483647의 범위를 가짐. UNSIGNED를 조건으로 줄 시 0~4294967295를 가진다. (2의 32제곱, 32bit, 4Bytes)
- **BIGINT** : INT보다 2제곱 많이, 즉, 2의 64제곱의 수를 대상으로 하며 또한 UNSIGNED 조건이 존재한다.(64bit, 8Bytes)
- **MEDIUMINT** : -8388608~8388607 사이의 수를 대상으로 하며 또한 UNSIGNED(0~16777215) 조건이 존재한다.(24bit, 3Bytes)
- **SMALLINT** : -32768~32767 사이의 수를 대상으로 하며 또한 UNSIGNED(0~65535) 조건이 존재한다.(16bit, 2Bytes)
- **TINYINT** : -128~127 사이의 수를 대상으로 하며 또한 UNSIGNED(0~255) 조건이 존재한다.(8bit, 1Bytes)
- **BOOL** : True와 False만 저장할 때 사용하는 값. 다만, 그렇다고 1bit를 사용하지 않습니다. TINYINT와 같이 1Bytes를 저장할 수 있지만 0이 아닌 모든 수는 True에 해당하므로 주의해야 합니다.
- 너비로 저장값에 제한을 줄 수 있으나 데이터 저장을 덜하게 만들진 않습니다.
Ex) 10만을 저장하기 위해 INT를 너비를 제한하더라도 저장은 4Bytes가 된다.

데이터를 조작하는 SQL : 자료형

▶ 실수형(부동 소수점)

- **FLOAT** : 총 **4Bytes**의 수를 이용하여 소수점 이하 **7자리**까지의 정밀도를 제공하는 소수점을 나타내는 타입. **7자리**가 넘어가는 수를 입력할 경우 나머지 자리수는 버린다. 근사값이므로 실제 값과 차이가 발생할 수 있다.
- **DOUBLE**: 총 **8Bytes**의 수를 이용하여 소수점 이하 **15자리**까지의 정밀도를 제공하는 소수점을 나타내는 타입. 이하 **FLOAT**와 같다.

▶ 실수형(고정 소수점)

- **DECIMAL**[(최대길이, 소수점길이)]
: 가변적인 형태의 용량을 저장하는 **5~17Bytes**의 타입. 최대 자리수(**65**)에서 소수점 자리수(최대 **30**)을 지정하여 고정적으로 소수점을 지정할 때 사용한다. 부동소수점과 다르게 근사값이 아니기 때문에 정확한 값을 저장할 수 있으나 정확도가 높아질수록 **Bytes**수가 증가하므로 주의해야한다. 때문에 생성시 총 자리수와 소수점 이하의 자리수를 각각 지정해줘야 한다. **Default**값은 **(10, 0)**[**10**자리수 중 소수점 이하는 **0**개]이다. 정수값은 자리수를 넘어갈 경우 에러, 소수점은 자리수를 넘어갈 경우 입력한 자리수 앞에서 반올림 된다.

데이터를 조작하는 SQL : 자료형

▶ 문자열 타입

- **CHAR(M) [CHARACTER SET {Set명}]** : 문자열 데이터타입으로, 입력할 데이터셋(utf8, utf16과 같은)과 길이에 따라 크기가 달라진다. **CHAR**타입은 고정 길이이므로 그보다 작은 글자는 공백이 채워진다. 최대 255글자까지 가능하다.
- **VARCHAR(M) [CHARACTER SET {Set명}]** : **CHAR**와 같지만 최대 65535길이의 글자까지 가능하다. 또한 **CHAR**와 다르게 고정길이가 아닌 가변길이를 위해 1~2Bytes의 길이를 나타내는 추가 용량이 필요하다.
- **BINARY / VARBINARY(M)** : 입력 방식은 위와 동일하지만 저장되는 방식은 바이트코드로 저장된다.
- **BLOB / TEXT [(M)]** : 대용량 문자열을 저장하는 데이터타입. **BLOB**은 바이트코드로 저장이 되고 **TEXT**는 순수 문자열로 저장이 된다.
- **TINYBLOB / TINYTEXT // MEDIUMBLOB / MEDIUMTEXT // LONGBLOB / LONGTEXT**
: 필요 길이에 따라 조절

데이터를 조작하는 SQL : 자료형

▶ 날짜 시간 타입

➤ DATE 타입

: YYYY-MM-DD, YY-MM-DD, YYYYMMDD, YYMMDD 등등의 방식으로 날짜타입을 입력하여 사용한다. 날짜만 저장할 때 사용한다. (1000-01-01 ~ 9999-12-31)

➤ TIME 타입

: HH:MM:SS 등등의 방식으로 시간타입을 입력하여 사용한다. 시간단위를 저장하며, 24시간제 이상의 시간도 저장이 가능하다. (-838:59:59 ~ 838:59:59)

➤ TIMESTAMP

: 1970-01-01 00:00:01부터 2038-01-19 03:14:07 까지 범위를 가지는 날짜와 시간을 합친 시간타입. UNIX 시간을 이용할 수 있다. UTC시간을 적용할 때 편하다. 소수점 이하에 대한 조건을 추가할 경우 소수점 이하 6자리의 정밀도를 추가할 수 있다.

➤ DATETIME

: 1000-01-01 00:00:00 ~ 9999-12-31 23:59:59의 범위를 가진다.

데이터를 조작하는 SQL : DELETE

- ▶ `DELETE FROM {Table명} [WHERE {조건식}] [ORDER BY {속성명} [추가 조건]] [LIMIT {INT}];`
- ▶ `DELETE FROM EMP WHERE ENAME IS NULL;`
- ▶ `START TRANSACTION;` (AUTO COMMIT을 잠시 비활성)
`DELETE FROM EMP;`
`SELECT * FROM EMP;`
`ROLLBACK;`
: 조건없이 사용하게 되면 큰일 날 수 있음.
- ▶ `START TRANSACTION;`
`DELETE FROM EMP WHERE ENAME = 'SAM';`
`COMMIT;`
: COMMIT을 해야만 적용이 됨. 실수를 예방할 수 있음. 기본적으로 **AUTO COMMIT**이 적용되어있음.
- ▶ 이 때, **COMMIT**과 **ROLLBACK**은 **DML**이 아니라, **TCL**에 해당한다.

데이터를 조작하는 SQL : UPDATE

- ▶ UPDATE {Table명} SET {적용 속성명} = {변경 값} [조건식];
: UPDATE EMP SET COMM = 100 WHERE COMM IS NULL;
: UPDATE EMP SET COMM = NULL WHERE COMM = 100;
- ▶ START TRANSACTION;
UPDATE EMP SET COMM = 100;
SELECT * FROM EMP;
ROLLBACK;
: UPDATE 또한 DELETE와 마찬가지로 조건을 잘 지정해 줘야한다.

CRUD를 통해 이전 예제 이해하기

- ▶ CREATE TABLE DEPT (DEPTNO DECIMAL(2), DNAME VARCHAR(14), LOC VARCHAR(13), CONSTRAINT PK_DEPT PRIMARY KEY (DEPTNO));
- ▶ INSERT INTO DEPT VALUES (10,'ACCOUNTING','NEW YORK'), (20,'RESEARCH','DALLAS'), (30,'SALES','CHICAGO'), (40,'OPERATIONS','BOSTON');
- ▶ CREATE TABLE EMP (EMPNO DECIMAL(4), ENAME VARCHAR(10), JOB VARCHAR(9), MGR DECIMAL(4), HIREDATE DATE, SAL DECIMAL(7,2), COMM DECIMAL(7,2), DEPTNO DECIMAL(2), CONSTRAINT PK_EMP PRIMARY KEY (EMPNO), CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO));
- ▶ INSERT INTO EMP VALUES (7369,'SMITH','CLERK',7902,'1980-12-17',800,NULL,20), (7499,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30), (7521,'WARD','SALESMAN',7698,'1981-02-21',1250,500,30), (7566,'JONES','MANAGER',7839,'1981-04-02',2975,NULL,20), (7654,'MARTIN','SALESMAN',7698,'1981-09-28',1250,1400,30), (7698,'BLAKE','MANAGER',7839,'1981-05-01',2850,NULL,30), (7782,'CLARK','MANAGER',7839,'1981-06-09',2450,NULL,10), (7788,'SCOTT','ANALYST',7566,'1987-07-13',3000,NULL,20), (7839,'KING','PRESIDENT',NULL,'1981-11-17',5000,NULL,10), (7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30), (7876,'ADAMS','CLERK',7788,'1987-07-13',1100,NULL,20), (7900,'JAMES','CLERK',7698,'1981-12-03',950,NULL,30), (7902,'FORD','ANALYST',7566,'1981-12-03',3000,NULL,20), (7934,'MILLER','CLERK',7782,'1982-01-23',1300,NULL,10);

Constraints?

- ▶ 하지만 모르는 부분이 존재합니다.
- ▶ **CONSTRAINT**라는 부분이 있는데 이는, 우리가 테이블을 관리함에 있어서 중요한 '제약조건'에 해당합니다.
- ▶ 만약, '이런 조건이면 데이터를 받지마!'라고 하려면 우리는 이를 로직으로 구현해야 합니다.
- ▶ 그런데 만약에 사전에 그런 조건을 '걸어만두어도' 자동으로 그 조건을 사용하게 만들면 어떨까요?
- ▶ 바로 제약조건에 대해서 알아보도록 하겠습니다.